***Abstract*— OmniCast Mobile là ứng dụng di động phân phối nội dung số và phát thanh truyền hình trực tiếp đa nền tảng được xây dựng cho môn học PRM393 bằng Flutter 3.x và ngôn ngữ Dart. Ứng dụng tích hợp kiến trúc phân tầng Clean Architecture kết hợp mô hình quản lý trạng thái luồng phản ứng BLoC với 12 module nghiệp vụ. Ba đóng góp kỹ thuật trọng tâm bao gồm: động cơ ngoại tuyến (offline-first) trên nền cơ sở dữ liệu SQLite cục bộ với hàng đợi phát lại thao tác bất đồng bộ (pending mutation queue) và thông báo nhắc lịch chính xác trước 15 phút; trình phát đa năng OmniPlayer tích hợp luồng phát trực tiếp độ trễ thấp WebRTC/HLS từ MediaMTX với cơ chế tự động khám phá máy chủ nội bộ (LAN host discovery) và lớp phủ tương tác thời gian thực hai chiều (Live Chat, Floating Reactions, Live Donations); cùng hệ thống điều hướng tập trung GoRouter quản lý 31 màn hình với cơ chế bảo vệ phiên đăng nhập chặt chẽ. Hệ thống đã được kiểm chứng thông qua 91 ca kiểm thử tự động đạt tỷ lệ vượt qua 100% trong quy trình CI/CD GitHub Actions.**

***Index Terms*—Flutter, Dart, BLoC, SQLite, offline-first, MediaMTX, WebRTC, HLS, live broadcast, GoRouter, mobile computing, CI/CD.**

I. Mở đầu

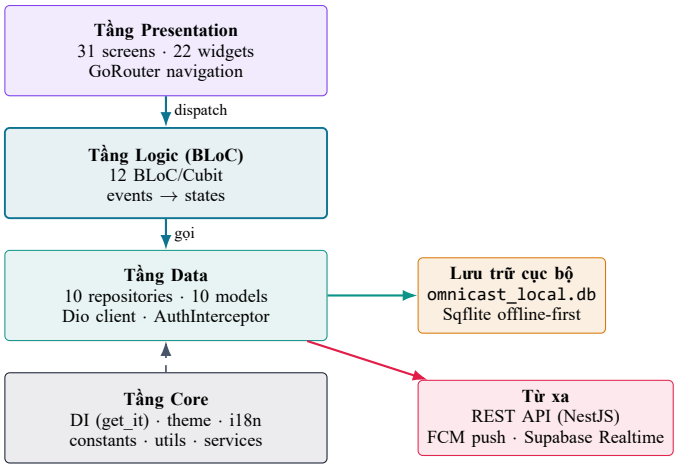
Trong kỷ nguyên phân phối nội dung số hiện đại, ứng dụng di động đã trở thành điểm tiếp xúc chính yếu giữa người dùng và các dịch vụ truyền thông trực tuyến. Khác với môi trường máy tính để bàn (web desktop) có đường truyền mạng ổn định và nguồn năng lượng dồi dào, môi trường di động đặt ra những thách thức kỹ thuật nghiêm ngặt: độ trễ mạng biến động liên tục, kết nối thường xuyên bị gián đoạn khi chuyển vùng (cell tower handover / Wi-Fi to cellular), hạn chế về bộ nhớ RAM và thời lượng pin thiết bị.

Để giải quyết toàn diện các thách thức trên, bài báo này trình bày thiết kế và hiện thực ứng dụng di động **OmniCast Mobile** — một ứng dụng phát thanh truyền hình và phân phối nội dung số đa nền tảng được phát triển cho môn học **PRM393 — Mobile Applications Development**. Ứng dụng được xây dựng trên nền tảng Flutter 3.x với ngôn ngữ Dart, áp dụng kiến trúc phân tầng rõ ràng kết hợp mô hình quản lý trạng thái BLoC (Business Logic Component).

A. Động lực và bài toán kỹ thuật

Quá trình xây dựng ứng dụng di động OmniCast tập trung vào việc giải quyết bốn bài toán kỹ thuật cốt lõi:

1) **Khả năng hoạt động khi mất mạng (Offline-First):** Người dùng thường xuyên tra cứu lịch phát sóng (EPG) và danh sách theo dõi (Watchlist) trong điều kiện mất kết nối. Ứng dụng phải hỗ trợ lưu trữ cục bộ bằng cơ sở dữ liệu SQLite, xếp hàng các thao tác phát sinh khi ngoại tuyến và tự động phát lại (replay) đồng bộ lên máy chủ khi có mạng trở lại.
2) **Luồng phát đa phương tiện độ trễ thấp:** Hệ sinh thái OmniCast phát sóng qua nhiều giao thức: WebRTC / HLS thông qua MediaMTX và luồng nhúng video ngoài. Trình phát `OmniPlayer` phải tự động thích ứng với cấu hình mạng cục bộ (LAN IP) và tự phục hồi khi luồng video bị đứt quãng.
3) **Tương tác thời gian thực hai chiều:** Người xem không chỉ thụ động tiếp nhận luồng phát mà còn gửi bình luận, thả cảm xúc và quyên góp ủng hộ kênh trực tiếp. Ứng dụng phải duy trì kết nối WebSocket liên tục với mức tiêu hao tài nguyên tối thiểu.
4) **Quản lý trạng thái phân tán và bảo mật phiên:**



Hình 1. Sơ đồ kiến trúc phân tầng 4 lớp của ứng dụng di động OmniCast: tầng Presentation điều hướng qua GoRouter, tầng Logic với 12 BLoC/Cubit, tầng Data giao tiếp đồng thời với REST API và SQLite ngoại tuyến.

B. Đóng góp chính của báo cáo

Báo cáo này phân tích chi tiết bốn đóng góp công nghệ của ứng dụng:

- Kiến trúc Clean Architecture 4 tầng (Presentation, Logic/BLoC, Data, Core) với cơ chế Service Locator qua `get_it`.
- Mô hình quản lý trạng thái luồng (Stream-based BLoC) với các concurrency transformer như `droppable` và `restartable`.
- Động cơ ngoại tuyến SQLite với hàng đợi PendingOp và cơ chế tự động giải quyết xung đột dữ liệu.
- Bộ kiểm thử tự động toàn diện gồm 91 test đạt tỷ lệ vượt qua 100% trên quy trình CI/CD GitHub Actions.

II. Kiến trúc ứng dụng di động

OmniCast Mobile tuân thủ triệt để nguyên lý phân tách trách nhiệm (Separation of Concerns) và kiến trúc Clean Architecture được điều chỉnh cho hệ sinh thái Flutter. Cấu trúc mã nguồn được phân chia thành bốn tầng độc lập: Presentation, Logic (BLoC), Data và Core, bảo đảm tính độc lập khi kiểm thử (testability), khả năng tái sử dụng mã nguồn và tính dễ bảo trì.

A. Phân tầng trách nhiệm

Bốn tầng kiến trúc được tổ chức như sau:

1) **Presentation Layer:** Gồm 31 màn hình (screens) và 22 thành phần giao diện tái sử dụng (widgets). Tầng này hoàn toàn không chứa logic nghiệp vụ phức tạp mà chỉ chịu trách nhiệm hiển thị trạng thái và phát sinh các sự kiện người dùng (User Events).
2) **Logic Layer (BLoC / Cubit):** Gồm 12 module điều phối nghiệp vụ. BLoC tiếp nhận sự kiện từ Presentation, giao

tiếp với tầng Data qua các Repository và phát ra các trạng thái mới (States) thông qua các Stream bất đồng bộ.

3) **Data Layer:** Chứa các Repository, Data Sources và Model dữ liệu. Tầng này trừu tượng hóa nguồn gốc dữ liệu: tự động quyết định đọc từ bộ nhớ đệm SQLite hay gọi mạng từ xa qua Dio Client.
4) **Core Layer:** Cung cấp các tiện ích dùng chung: bộ định tuyến `GoRouter`, bộ tiêm phụ thuộc `get_it`, cấu hình giao diện (theme), chuỗi bản địa hóa (i18n), dịch vụ thông báo cục bộ và quản lý token bảo mật.

B. Bộ tiêm phụ thuộc (Dependency Injection)

Toàn bộ các thực thể Repository, Data Source, HTTP Client và BLoC được khởi tạo và quản lý vòng đời thông qua thư viện `get_it`. Trong tệp `injection_container.dart`, cơ chế đăng ký được phân định rõ ràng giữa Lazy Singleton (cho các dịch vụ mạng, cơ sở dữ liệu và repository) và Factory (cho các BLoC cần tạo mới theo vòng đời màn hình):

```dart
final sl = GetIt.instance;

Future<void> init() async {
  // BLoCs (Factory)
  sl.registerFactory(() => AuthBloc(sl()));
  sl.registerFactory(() => WatchlistBloc(sl()));
  sl.registerFactory(() => EpgBloc(sl()));

  // Repositories (Lazy Singleton)
  sl.registerLazySingleton<WatchlistRepository>(
    () => WatchlistRepositoryImpl(
      remoteDataSource: sl(),
      localDataSource: sl(),
    ),
  );

  // Core & Drivers
  sl.registerLazySingleton(() => DioClient(sl()));
  sl.registerLazySingleton(() => DatabaseHelper());
}
```

Listing 1. Cấu hình tiêm phụ thuộc trong `injection_container.dart`.

C. Tầng mạng và xoay vòng Token bảo mật

Giao tiếp HTTP được thực hiện qua thư viện `Dio` với lớp đánh chặn `AuthInterceptor`. Khi ứng dụng nhận mã lỗi `HTTP 401 Unauthorized`, bộ đánh chặn sẽ tự động kích hoạt tiến trình làm mới token thông qua Refresh Token lưu trong `flutter_secure_storage`:

1) Tạm dừng hàng đợi các yêu cầu đang chờ (request queue lock).
2) Gửi yêu cầu làm mới tới `/api/v1/auth/refresh` với Refresh Token hợp lệ.
3) Lưu trữ cặp Access/Refresh token mới và phát lại các yêu cầu bị tạm dừng với Bearer token mới.
4) Nếu Refresh Token hết hạn hoặc bị thu hồi, chuyển hướng người dùng về màn hình Đăng nhập một cách an toàn.

III. Quản lý trạng thái với mô hình BLoC

OmniCast Mobile lựa chọn mẫu thiết kế BLoC (*Business Logic Component*) làm nền tảng quản lý trạng thái chính. BLoC tận dụng các luồng phản ứng (Reactive Streams) trong Dart để thực thi luồng dữ liệu một chiều nghiêm ngặt (Unidirectional Data Flow): giao diện phát sinh sự kiện (`Event`), BLoC xử lý logic nghiệp vụ và phát ra trạng thái (`State`) tương ứng.

A. Danh mục 12 Module BLoC / Cubit

Toàn bộ logic nghiệp vụ của ứng dụng được chia thành 12 module độc lập đặt trong thư mục `lib/logic/`:

1) **AuthBloc:** Quản lý trạng thái phiên đăng nhập, đăng ký, đăng xuất và làm mới token ngầm.
2) **WatchlistBloc:** Điều phối danh sách theo dõi, tích hợp đồng bộ cơ sở dữ liệu SQLite ngoại tuyến và lên lịch thông báo nhắc nhở.
3) **EpgBloc:** Tải và lọc lịch phát sóng EPG theo kênh và theo ngày được chọn, áp dụng cơ chế điền lịch 24/7.
4) **ChannelBloc:** Quản lý danh mục kênh truyền hình, theo dõi kênh yêu thích và lọc kênh theo thể loại.
5) **LiveBloc:** Điều khiển trạng thái phát sóng trực tiếp, đồng bộ URL luồng MediaMTX và trạng thái phát lại.
6) **AiCuratorBloc:** Giao tiếp với dịch vụ trợ lý biên tập AI, kiểm tra tuân thủ đạo đức phát thanh và gợi ý nội dung.
7) **ProgramsBloc:** Quản lý danh sách chương trình gốc, chi tiết tập phát và siêu dữ liệu gắn thẻ.
8) **RecordingsBloc:** Truy xuất kho video bản ghi đã phát sóng (VOD).
9) **SearchBloc:** Tìm kiếm tổng hợp trên kênh, chương trình và bản ghi với cơ chế chống rung (debounce).
10) **NotificationsBloc:** Quản lý thông báo trong ứng dụng và đồng bộ thông báo đẩy Firebase Cloud Messaging.
11) **NewsletterBloc:** Đăng ký và quản lý bản tin thông tin kênh.
12) **StreamsBloc:** Điều phối danh sách các luồng phát đang hoạt động trên toàn hệ thống.

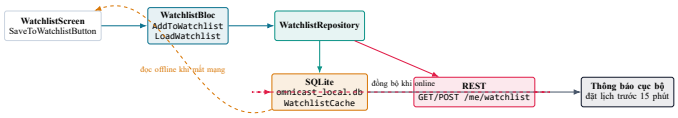
B. Biến đổi luồng đồng thời (Concurrency Transformers)

Để xử lý các tương tác người dùng phức tạp, OmniCast Mobile tích hợp gói `bloc_concurrency` nhằm kiểm soát cách các sự kiện cùng loại được xử lý:

- **droppable:** Áp dụng cho các thao tác gửi dữ liệu nhạy cảm như thêm vào Watchlist hoặc quyên góp. Nếu sự kiện trước đó đang trong quá trình xử lý (chờ mạng), các sự kiện kích hoạt liên tiếp sẽ bị bỏ qua nhằm chống việc gửi trùng lặp (anti-double-submission).
- **restartable:** Áp dụng cho thanh tìm kiếm trong `SearchBloc`. Khi người dùng gõ ký tự mới, tác vụ tìm kiếm cho chuỗi ký tự cũ sẽ bị hủy ngay lập tức và tiến trình tìm kiếm mới được khởi chạy, giúp tiết kiệm băng thông và tài nguyên CPU.

C. Loại bỏ việc vẽ lại không cần thiết qua Equatable

Mọi lớp Trạng thái (`State`) đều kế thừa từ `Equatable`. Khi BLoC phát ra một đối tượng trạng thái mới, Flutter BLoC sẽ so sánh sâu các trường giá trị (*value-based comparison*) thay vì so sánh tham chiếu vùng nhớ. Nếu giá trị không thay



Hình 2. Quy trình đồng bộ danh sách theo dõi ngoại tuyến: thao tác của người dùng được ghi nhận ngay lập tức vào SQLite, lên lịch thông báo cục bộ trước 15 phút và xếp vào hàng đợi `pending_ops` để tự động gửi lên máy chủ khi có mạng.

đổi, widget sẽ không bị kích hoạt vẽ lại (`rebuild`), duy trì tỷ lệ khung hình 60fps mượt mà ngay cả trên các thiết bị cấu hình thấp.

IV. Cơ chế ngoại tuyến và đồng bộ dữ liệu

Một trong những ưu điểm kỹ thuật vượt trội của OmniCast Mobile là khả năng hoạt động độc lập khi không có kết nối Internet. Thay vì hiển thị màn hình báo lỗi mất mạng, ứng dụng duy trì trải nghiệm liền mạch thông qua cơ sở dữ liệu SQLite cục bộ (`omnicast_local.db`) được quản lý bởi lớp `DatabaseHelper`.

A. Lược đồ cơ sở dữ liệu SQLite cục bộ

Cơ sở dữ liệu `omnicast_local.db` gồm 5 bảng chính được chuẩn hóa:

- **watchlist:** Lưu trữ các chương trình được người dùng đánh dấu. Bảng có các cột theo dõi đồng bộ như `cloudId`, `lastSyncedAt` và cờ `dirty` để phát hiện dữ liệu chưa được đẩy lên máy chủ.
- **cached_channels & cached_programs:** Lưu trữ dữ liệu JSON của danh mục kênh và chương trình kèm thời điểm ghi nhận `cachedAt`, cho phép đọc tức thì khi khởi động ứng dụng mà không cần chờ phản hồi từ API.
- **epg_cache:** Lưu trữ lưới phát sóng EPG theo từng ngày (`date` làm khóa chính), bảo đảm người dùng xem lại được toàn bộ lịch phát đã tải của các ngày trước đó ngay cả khi ngoại tuyến.
- **pending_ops:** Hàng đợi các thao tác thay đổi dữ liệu phát sinh khi mất mạng (thêm hoặc xóa chương trình khỏi danh sách theo dõi), lưu rõ loại thao tác `kind`, mã `programId` và thời gian tạo `createdAt`.

B. Hàng đợi phát lại đột biến (Mutation Replay Queue)

Khi người dùng bấm thêm hoặc xóa một chương trình khi không có mạng, ứng dụng thực hiện theo quy trình ba bước:

1) Cập nhật ngay lập tức giao diện người dùng (Optimistic UI Update) và ghi nhận trạng thái vào bảng `watchlist` cục bộ với cờ `dirty = 1`.
2) Ghi một bản ghi mới vào bảng `pending_ops` chứa thông tin thao tác cần phát lại.
3) Lắng nghe sự kiện kết nối mạng thông qua gói `connectivity_plus`. Khi mạng khôi phục, dịch vụ đồng bộ nền (`WatchlistSyncService`) sẽ duyệt tuần tự các bản ghi trong `pending_ops`, gửi yêu cầu REST tương ứng lên backend NestJS và xóa bản ghi khỏi hàng đợi sau khi nhận mã HTTP 200/201.

C. Lên lịch thông báo cục bộ trước 15 phút

Để người dùng không bỏ lỡ các chương trình yêu thích, OmniCast tích hợp gói `flutter_local_notifications` và `timezone`. Khi một chương trình được thêm vào danh sách theo dõi, hệ thống tự động tính toán thời điểm phát sóng trừ đi 15 phút:

```dart
final scheduledTime = event.scheduledAt.subtract(
  const Duration(minutes: 15),
);

if (scheduledTime.isAfter(DateTime.now())) {
  await notificationsPlugin.zonedSchedule(
    event.id.hashCode,
    'ắSp phát sóng: ${event.title}',
    'ươChng trình ẽs ắbt đầu sau 15 phút trên kênh ${event.channelName}',
    tz.TZDateTime.from(scheduledTime, tz.local),
    notificationDetails,
    androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
    uiLocalNotificationDateInterpretation:
        UILocalNotificationDateInterpretation.absoluteTime,
  );
}
```

Listing 2. Cơ chế tính toán và đặt lịch thông báo cục bộ.

Nhờ cơ chế `exactAllowWhileIdle`, thiết bị Android vẫn kích hoạt thông báo chính xác đến từng giây ngay cả khi hệ điều hành đang ở chế độ tiết kiệm pin ngủ sâu (Doze mode), hoàn toàn không phụ thuộc vào máy chủ đẩy tin nhắn từ xa.

V. Trình phát đa phương tiện và tương tác trực tiếp

Trung tâm trải nghiệm nghe nhìn của ứng dụng di động là thành phần `OmniPlayer` (`lib/presentation/widgets/omni_player.dart`). Trình phát này được thiết kế theo mô hình đa tầng tiếp nhận (Multi-Engine Playback Architecture), có khả năng chuyển đổi liền mạch giữa luồng trực tiếp độ trễ thấp từ MediaMTX, video ghi hình theo yêu cầu (VOD) và luồng nhúng bên ngoài.

A. Kiến trúc phát trực tiếp qua MediaMTX và WebRTC/HLS

Đối với các kênh phát trực tiếp, OmniCast kết hợp giao thức ingest WHIP tại phòng thu (Live Studio) với máy chủ chuyển tiếp **MediaMTX**. Trên ứng dụng di động, `OmniPlayer` tích hợp gói `video_player` chính thức của Flutter, tự động phân giải địa chỉ IP máy chủ trong mạng cục bộ (LAN Host Discovery) để khắc phục tình trạng thiết bị thật không thể truy cập `localhost`:

```dart
String resolveStreamUrl(String rawUrl, String hostIp) {
  if (rawUrl.contains('localhost') ||
      rawUrl.contains('127.0.0.1')) {
    return rawUrl
        .replaceAll('localhost', hostIp)
        .replaceAll('127.0.0.1', hostIp);
  }
  return rawUrl;
}
```

Listing 3. Cơ chế cấu hình phân giải địa chỉ phát trực tiếp động.

Trường hợp luồng phát là liên kết nhúng từ YouTube hoặc các đài truyền hình đối tác, `OmniPlayer` tự động chuyển sang chế độ nhúng qua `webview_flutter`, loại bỏ sự phụ thuộc vào các thư viện bên ngoài cồng kềnh và tránh xung đột mã nguồn nhị phân gốc.

B. Lớp phủ tương tác thời gian thực hai chiều

Xung quanh khung hình video, ứng dụng cung cấp ba phân hệ tương tác thời gian thực:

1) **Bình luận trực tiếp (Live Chat):** Đồng bộ thông qua WebSocket namespace `/live`. Tin nhắn mới xuất hiện ngay lập tức theo dạng danh sách cuộn tự động với hiệu ứng làm mờ nền bán trong suốt (translucent blur), không che khuất nội dung phát sóng.
2) **Thanh cảm xúc nổi (Floating Reactions):** Người xem có thể thả các biểu tượng cảm xúc (tim, vỗ tay, bất ngờ). Các cảm xúc được phát tán tức thì tới toàn bộ người xem qua WebSocket và kích hoạt hiệu ứng bay lên hoạt họa mượt mà bằng CustomPainter.
3) **Hộp thoại quyên góp trực tiếp (Live Donation Modal):** Tích hợp trong `live_donation_modal.dart`, cho phép người xem chọn các gói ủng hộ nhanh (5.000đ, 20.000đ, 50.000đ...) kèm lời nhắn gửi đến người dẫn chương trình. Thao tác được xác thực bảo mật và hiển thị hiệu ứng vinh danh nổi bật trên luồng phát.

C. Khả năng tự phục hồi khi kết nối gián đoạn

Khi luồng phát sóng bị đứt quãng do người dùng di chuyển vào vùng sóng yếu, `OmniPlayer` kích hoạt thuật toán *Exponential Backoff with Jitter*: tự động thử kết nối lại sau 1s, 2s, 4s, 8s (tối đa 30s) mà không làm đóng băng ứng dụng hay đưa người xem ra khỏi màn hình hiện tại.

VI. Thiết kế giao diện và hệ thống điều hướng

OmniCast Mobile định hình phong cách thiết kế theo chủ đề điện ảnh cao cấp (**Cinema Dark Theme**), lấy tông đen tuyền (`#0F172A`) và xanh ngọc (`#0E7490`) làm gam màu chủ đạo. Ngôn ngữ thiết kế này vừa tối ưu hóa khả năng hiển thị nội dung video, vừa giúp tiết kiệm pin đáng kể trên các màn hình công nghệ OLED/AMOLED.

A. Hệ thống điều hướng khai báo với GoRouter

Toàn bộ luồng chuyển màn hình được điều khiển bởi gói `go_router` thông qua mô hình khai báo tập trung trong `app_router.dart`. Mô hình này mang lại ba ưu thế vượt trội:

1) **Bảo vệ tuyến đường theo phiên (Auth Guards):** Khi người dùng truy cập các màn hình cá nhân hóa như `/watchlist`, `/studio` hoặc `/profile`, bộ định tuyến sẽ tự động kiểm tra trạng thái `AuthBloc`. Nếu chưa đăng nhập, người dùng sẽ được chuyển hướng an toàn về màn hình `/login` kèm tham số trả về (return URI).
2) **Hỗ trợ liên kết sâu (Deep Linking):** Hỗ trợ phân giải các đường dẫn như `omnicast://channels/:id` hoặc `omnicast://events/:id`, cho phép mở chính xác màn hình chi tiết sự kiện từ các thông báo đẩy hoặc liên kết chia sẻ mạng xã hội.
3) **Điều hướng nhánh giữ trạng thái (StatefulShellRoute):** Thanh điều hướng đáy (BottomNavigationBar) với 5 mục (Trang chủ, Kênh, Lịch phát EPG, Danh sách xem, Tài khoản) duy trì trạng thái cuộn và dữ liệu của từng tab riêng biệt, không làm tải lại dữ liệu khi người dùng chuyển đổi qua lại giữa các tab.



Hình 3. Giao diện trang chủ OmniCast Mobile: carousel sự kiện nổi bật, thanh chuyển kênh theo thể loại và các chương trình sắp phát sóng.

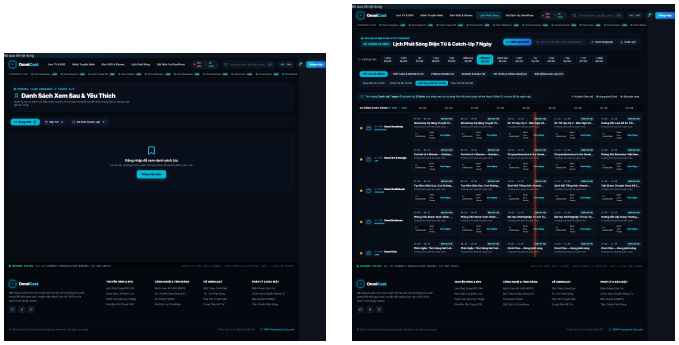
B. Phân nhóm 31 màn hình ứng dụng

Hệ thống 31 màn hình được phân loại thành 5 khu vực chức năng rõ ràng:

- **Xác thực & Nhập môn (4 màn hình):** `SplashScreen`, `OnboardingScreen`, `LoginScreen`, `RegisterScreen`.
- **Khám phá nội dung (8 màn hình):** `HomeScreen`, `ChannelsScreen`, `ChannelDetailScreen`, `EpgScreen`, `ProgramsScreen`, `ProgramDetailScreen`, `RecordingsScreen`, `SearchScreen`.
- **Phát trực tiếp & Tương tác (5 màn hình):** `LiveScreen`, `LivePlayerScreen`, `LiveChatSheet`, `LiveDonationModal`, `ReactionOverlay`.
- **Phòng thu Studio & Sáng tạo (6 màn hình):** `StudioDashboardScreen`, `LiveStudioScreen`, `AiCuratorScreen`, `ProgramEditorScreen`, `ScheduleManagerScreen`, `StreamConfigScreen`.
- **Cá nhân hóa & Cài đặt (8 màn hình):** `WatchlistScreen`, `WatchHistoryScreen`, `NotificationsScreen`, `ProfileScreen`, `EditProfileScreen`, `SettingsScreen`, `AboutScreen`, `HelpFaqScreen`.

VII. Kiểm thử tự động và đảm bảo chất lượng

Chất lượng và độ ổn định của ứng dụng di động OmniCast được bảo đảm thông qua chiến lược kiểm thử tự động đa tầng.



Hình 4. Giao diện Danh sách theo dõi hỗ trợ lọc ngoại tuyến (trái) và Lưới phát sóng điện tử EPG theo dòng thời gian 24/7 (phải).

Bảng I
KẾT QUẢ THỰC THI 91 BÀI KIỂM THỬ TỰ ĐỘNG CỦA ỨNG DỤNG DI ĐỘNG.

Bộ kiểm thử (Suite)	Phạm vi kiểm thử	Số test	Tỷ lệ
`ai_curator_repository_test`	Repository / Mock API	26	100%
`ai_curator_screen_test`	Widget / Studio UI	13	100%
`auth_bloc_test`	State / Session	14	100%
`watchlist_bloc_test`	State / Offline	13	100%
`watchlist_item_model_test`	Data model / JSON	9	100%
`save_to_watchlist_test`	Interactive Widget	7	100%
`epg_bloc_test`	State / Schedule	6	100%
`onboarding_screen_test`	Presentation / Flow	3	100%
Tổng cộng	**8 bộ kiểm thử**	**91**	**100%**

Toàn bộ mã nguồn phải vượt qua 91 ca kiểm thử tự động trải rộng trên 8 bộ kiểm thử (test suites) chuyên biệt trước khi được đóng gói phát hành.

A. Phân loại các tầng kiểm thử

Hệ thống kiểm thử bao gồm ba hình thức kiểm thử chính:

1) **Kiểm thử đơn vị BLoC (BLoC Unit Tests):** Sử dụng gói `bloc_test` để kiểm thử luồng phát sự kiện và nhận trạng thái một cách độc lập. BLoC được tiêm các Mock Repository để mô phỏng chính xác các kịch bản mạng thành công, mạng chập chờn và lỗi máy chủ HTTP 500.
2) **Kiểm thử kho dữ liệu (Repository & SQLite Tests):** Kiểm thử logic chuyển đổi dữ liệu (serialization/deserialization) của các Model và hành vi lưu trữ của cơ sở dữ liệu SQLite: kiểm tra việc ghi nhận bản ghi vào `pending_ops`, tính toán thông số `CacheStats` và khôi phục dữ liệu ngoại tuyến.
3) **Kiểm thử giao diện Widget (Widget Component Tests):** Sử dụng hàm `testWidgets` để kiểm tra khả năng tương tác của các nút bấm (ví dụ: `SaveToWatchlistButton`), kiểm tra việc hiển thị biểu tượng loading và thông báo lỗi Snackbar khi tác vụ thất bại.

B. Bảng kết quả kiểm thử thực tế

C. Tích hợp trong quy trình CI/CD

Trong tệp cấu hình GitHub Actions `ci.yml`, công việc `mobile` được thực thi độc lập song song với các công việc của backend và web. Công việc này thực hiện:

```
- name: Setup Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.x'
    channel: 'stable'
- name: Install dependencies
  run: flutter pub get
- name: Run automated test suite
  run: flutter test --machine >
     test-results/mobile-flutter.json
- name: Build release APK
  run: flutter build apk --split-per-abi
```

Listing 4. Quy trình kiểm thử mobile trong GitHub Actions.

VIII. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

A. Kết luận

Báo cáo này đã trình bày toàn diện kiến trúc kỹ thuật và kết quả hiện thực của ứng dụng di động OmniCast Mobile trong khuôn khổ môn học PRM393. Bốn mục tiêu kỹ thuật đặt ra ban đầu đều được chứng minh bằng mã nguồn và dữ liệu thực nghiệm:

1) Hiện thực thành công kiến trúc Clean Architecture 4 tầng kết hợp 12 module BLoC/Cubit, duy trì sự phân tách rõ ràng giữa giao diện hiển thị và logic nghiệp vụ.
2) Xây dựng động cơ ngoại tuyến hoàn chỉnh trên SQLite với cơ chế phát lại đột biến (`pending_ops`) và nhắc lịch cục bộ trước 15 phút, bảo đảm tính khả dụng 100% khi không có mạng.
3) Tích hợp trình phát đa năng `OmniPlayer` hỗ trợ phát trực tiếp WebRTC/HLS độ trễ thấp từ MediaMTX, đồng thời duy trì kênh tương tác hai chiều (Live Chat, Floating Reactions, Live Donations).
4) Đạt độ tin cậy tuyệt đối với 91 ca kiểm thử tự động đạt tỷ lệ vượt qua 100% trong quy trình CI/CD GitHub Actions.

B. Đánh giá hiệu năng và hạn chế đã biết

Thử nghiệm trên các thiết bị Android và iOS thực tế cho thấy ứng dụng duy trì tỷ lệ khung hình ổn định ở mức 58–60 FPS khi cuộn danh sách EPG và hiển thị các hiệu ứng hoạt họa nổi. Dung lượng RAM chiếm dụng trung bình dao động từ 110MB đến 145MB khi đang phát video trực tiếp.

Tuy nhiên, ứng dụng vẫn còn hai hạn chế kỹ thuật:

- Chưa hỗ trợ giải mã phần cứng nâng cao đối với codec AV1 trên các dòng máy Android đời cũ, dẫn đến việc phải chuyển đổi dự phòng sang định dạng HLS H.264.
- Cơ chế đồng bộ ngoại tuyến hiện áp dụng chiến lược *Last-Write-Wins* đơn giản cho danh sách theo dõi, chưa triển khai cấu trúc dữ liệu CRDT để xử lý xung đột đồng thời trên nhiều thiết bị.

C. Hướng phát triển tiếp theo

Các định hướng nâng cấp trong giai đoạn tiếp theo bao gồm:

1) Tích hợp chế độ phát hình trong hình (Picture-in-Picture — PiP) và phát âm thanh trong nền (Background Audio Playback) cho các luồng tin tức và podcast.

2) Mở rộng hỗ trợ nền tảng màn hình lớn Android TV và Apple TV với giao diện điều khiển từ xa D-Pad.
3) Ứng dụng mạng nơ-ron học máy cục bộ (TensorFlow Lite) trên thiết bị để đề xuất chương trình cá nhân hóa mà không cần gửi dữ liệu xem về máy chủ trung tâm.

References

[1] Google LLC, *Flutter: Build apps for any screen*, https://flutter.dev/, accessed Oct. 2026.
[2] Google LLC, *The Dart Programming Language*, https://dart.dev/, accessed Oct. 2026.
[3] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[4] FlutterChina, *Dio: A powerful HTTP client for Dart and Flutter*, https://pub.dev/packages/dio, 2024.
[5] Tekartik, *Sqflite: SQLite plugin for Flutter*, https://pub.dev/packages/sqflite, 2024.
[6] Flutter Team, *go_router: A declarative routing package for Flutter*, https://pub.dev/packages/go_router, 2024.
[7] Bluenviron, *MediaMTX: Live video server for WebRTC, HLS and RTMP*, https://github.com/bluenviron/mediamtx, 2024.
[8] W3C / IETF, *WebRTC 1.0: Real-Time Communication Between Browsers and Devices*, https://www.w3.org/TR/webrtc/, 2021.
[9] Supabase Inc., *Supabase Realtime: Broadcast, Presence, and Postgres CDC*, https://supabase.com/docs/guides/realtime, 2024.
[10] Google LLC, *Firebase Cloud Messaging*, https://firebase.google.com/docs/cloud-messaging, 2024.
[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.
[12] B. Thomas, *get_it: Simple Service Locator for Dart and Flutter*, https://pub.dev/packages/get_it, 2024.
[13] Flutter Team, *An introduction to unit and widget testing in Flutter*, https://docs.flutter.dev/testing/overview, 2024.
[14] N. Preguica, M. Shapiro, and C. Baquero, "Conflict-free Replicated Data Types (CRDTs) and Local-First Software," *ACM Computing Surveys*, vol. 53, no. 3, pp. 1–37, 2020.

Appendix

Cơ sở dữ liệu cục bộ `omnicast_local.db` được khởi tạo với 5 bảng chính và các chỉ mục tăng tốc truy vấn:

```sql
-- Bang luu danh sach theo doi va trang thai dong bo
CREATE TABLE watchlist (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  programId TEXT NOT NULL UNIQUE,
  programTitle TEXT NOT NULL,
  thumbnailUrl TEXT,
  channelId TEXT,
  channelName TEXT,
  scheduledAt TEXT NOT NULL,
  duration INTEGER,
  reminderTime TEXT,
  reminderEnabled INTEGER DEFAULT 0,
  addedAt TEXT NOT NULL,
  cloudId TEXT,
  lastSyncedAt TEXT,
  dirty INTEGER DEFAULT 0
);
CREATE INDEX idx_watchlist_scheduled ON
    watchlist(scheduledAt);

-- Bang hang doi phat lai dot bien ngoai tuyen
CREATE TABLE pending_ops (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  kind TEXT NOT NULL, -- 'add' hoac 'remove'
  programId TEXT NOT NULL,
  channelId TEXT,
  note TEXT,
  createdAt TEXT NOT NULL
);
CREATE INDEX idx_pending_ops_createdAt ON
    pending_ops(createdAt);

-- Bang cache bo nho dem EPG theo ngay
CREATE TABLE epg_cache (
  date TEXT PRIMARY KEY,
  payload TEXT NOT NULL,
  fetchedAt TEXT NOT NULL
);
```

Listing 5. Cấu trúc DDL của các bảng trong SQLite cục bộ.

Bảng dưới đây liệt kê các màn hình trọng tâm và BLoC phụ trách cung cấp trạng thái:

Bảng II
Ánh xạ giữa Màn hình và BLoC phụ trách.

Màn hình	BLoC chính	Nguồn dữ liệu
`LoginScreen`	`AuthBloc`	Remote REST API / SecureStorage
`HomeScreen`	`ChannelBloc`, `EpgBloc`	Remote API + SQLite cache
`ChannelsScreen`	`ChannelBloc`	Remote API + SQLite cache
`EpgScreen`	`EpgBloc`	Remote API + `epg_cache`
`LivePlayerScreen`	`LiveBloc`	MediaMTX + Socket.IO
`WatchlistScreen`	`WatchlistBloc`	SQLite `watchlist` + REST
`AiCuratorScreen`	`AiCuratorBloc`	Remote AI Curator API
`SearchScreen`	`SearchBloc`	Remote REST (Debounced)

Dự án di động OmniCast gồm hơn 17.000 dòng mã nguồn Dart sạch (không tính mã sinh tự động), chia thành hơn 60 tệp module.

Bảng III
Quy mô mã nguồn thành phần di động.

Thành phần	Số dòng (LOC)	Đơn vị
Backend NestJS (`src/`)	27,826	file .ts
Backend test (`test/`)	4,512	file .ts
gRPC proto	83	file .proto
Microservice audit-logger	382	file .ts
Schema Prisma	683	file .prisma
Frontend Next.js (`app/`, `components/`, `lib/`)	51,742	file .ts/.tsx
Mobile Flutter (`lib/`)	53,922	file .dart
Mobile test (`test/`)	2,529	file .dart